CHAPTER 19

Beyond requirements development

The Chemical Tracking System's project sponsor, Gerhard, had been skeptical of the need to spend time defining requirements. However, he joined the development team and product champions at a one-day training class on software requirements, which motivated him to support the requirements activities.

As the project progressed, Gerhard received excellent feedback from the user representatives about how well requirements development had gone. He even sponsored a luncheon for the analysts and product champions to celebrate reaching the significant milestone of baselined requirements for the first system release. At the luncheon, Gerhard thanked the participants for their effective teamwork. Then he said, "Now that the requirements are done, I look forward to seeing the final product."

"Please keep in mind, Gerhard, we won't have the final product for about a year," the project manager explained. "We plan to deliver the system through a series of bimonthly releases. If we take the time to think about design now, it will be easier for developers to add more functionality later. We'll also learn more about requirements as we go along. We will show you some working software at each release, though."

Gerhard was frustrated. It looked like the development team was stalling rather than getting down to the real work of programming. But was he jumping the gun?

Experienced project managers and developers understand the value of translating software requirements into rational project plans and robust designs. These steps are necessary whether the next release represents 1 percent or 100 percent of the final product. This chapter explores some approaches for bridging the gap between requirements development and a successful product release. Some of these activities are the business analyst's responsibility, whereas others fall within the project manager's domain. We'll look at several ways in which requirements influence project plans, designs, code, and tests, as shown in Figure 19-1. In addition to these connections, there is a link between the requirements for the software to be built and other project and transition requirements. Those include data migrations, training design and delivery, business process and organizational changes, infrastructure modifications, and others. Those activities aren't discussed further in this book.

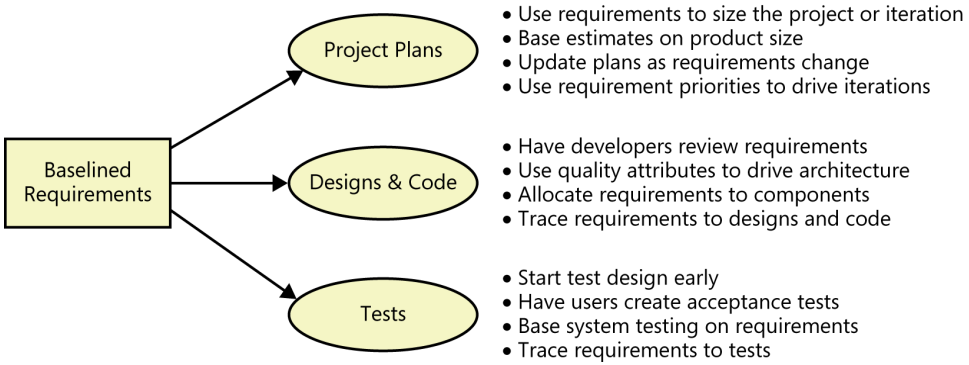

FIGURE 19-1 Requirements drive project planning, design, coding, and testing activities.

Estimating requirements effort

One of the earliest project planning activities is to judge how much of the project's schedule and effort should be devoted to requirements activities. Karl Wiegers (2006) suggests some ways to judge this and some factors that would lead you to spend either more or less time than you might otherwise expect. Small projects typically spend 15 to 18 percent of their total effort on requirements work (Wiegers 1996), but the appropriate percentage depends on the size and complexity of the project. Despite the fear that exploring requirements will slow down a project, considerable evidence shows that taking time to understand the requirements actually accelerates development, as the following examples illustrate:

- A study of 15 projects in the telecommunications and banking industries revealed that the most successful projects spent 28 percent of their resources on requirements elicitation, modeling, validation, and verification (Hofmann and Lehner 2001). The average project devoted 15.7 percent of its effort and 38.6 percent of its schedule to requirements engineering.
- NASA projects that invested more than 10 percent of their total resources on requirements development had substantially smaller cost and schedule overruns than projects that devoted less effort to requirements (Hooks and Farry 2001).
- In a European study, teams that developed products more quickly devoted more of their schedule and effort to requirements than did slower teams, as shown in Table 19-1 (Blackburn, Scudder, and Van Wassenhove 1996).

TABLE 19-1 Investing in requirements accelerates development

	Effort devoted to requirements	Schedule devoted to requirements
Faster projects	14%	17%
Slower projects	7%	9%

Requirements engineering activity is distributed throughout the project in different ways, depending on whether the project is following a sequential (waterfall), iterative, or incremental development life cycle, as was illustrated in Figure 3-3 in Chapter 3, "Good practices for requirements engineering."

> **Trap** Watch out for analysis paralysis. A project with massive up-front effort aimed at perfecting the requirements "once and for all" often delivers little useful functionality in an appropriate time frame. On the other hand, don't avoid requirements development because of the specter of analysis paralysis. As with so many issues in life, a sensible balance point lies somewhere between the two extremes.

When estimating the effort a project should devote to requirements development, let experience be your guide. Look back at the requirements effort from previous projects and judge how effective the requirements work on those projects was. If you can attribute issues to poor requirements, perhaps more emphasis on requirements work would pay off for you. Of course, this assessment demands that you retain some historical data from previous projects so you can better estimate future projects. You might not have any such data now, but if team members record how they spend their time on today's project, that becomes tomorrow's "historical data." It's not more complicated than that. Recording both estimated and actual effort allows you to think of how you can improve future estimates.

The requirements engineering consulting company Seilevel (Joy's company) developed an effective approach for estimating a project's requirements development effort, refined from work estimates and actual results from many projects. This approach involves three complementary estimates: percent of total work; a developer-to-BA ratio; and an activity breakdown that uses basic resource costs to generate a bottom-up estimate. Comparing the results from all three estimates and reconciling any significant disconnects allows the business analyst team to generate the most accurate estimates.

The first estimate is based on a percentage of the estimated total project work. Specifically, we consider that about 15 percent of the total project effort should be allocated to requirements work. This value is in line with the percentages cited earlier in this section. So if the full project is estimated at 1,000 hours, we estimate 150 hours of requirements work. Of course, the overall project estimate might change after the requirements are better understood.

The second type of estimate assumes a typical ratio of developers to business analysts. Our default value is 6:1, meaning that one BA can produce enough requirements to keep six developers busy. The BAs also will be working with quality assurance, project management, and the business itself, so this estimate encompasses all of the BA team's project work. For a packaged solution (COTS) project, this ratio changes to 3:1 (three developers per BA). There are still many selection, configuration, and transition requirements to be elicited and documented, but the development team is smaller because the code is largely purchased instead of developed new. So if we know the development team size, we can estimate an appropriate BA staffing level. This is a rule-of-thumb estimator, not a cast-in-concrete prediction of the future, so be sure to adjust for your organization and project type.

The third estimate considers the various activities a BA performs, based on estimates of the numbers of various artifacts that might be created on a specific project. The BA can estimate the number of process flows, user stories, screens, reports, and the like and then make reasonable assumptions of how many other requirements artifacts are needed. Based on time estimates per activity that we have accumulated from multiple projects, we can generate a total requirements effort estimate.

We created a spreadsheet tool for calculating all three of these requirements estimates, which is available with this book's companion content. Figure 19-2 illustrates a portion of the spreadsheet's results. The Summary Total Effort Comparison shows the estimates for the number of BAs and the BA budgets for both the requirements work and the entire project. These estimates serve as a starting point for reconciling the differences, negotiating resources, and planning the project's BA needs.

Input				
Quantity	Items	Quantity	Items	
20	Existing pages of documentation for review	$ 750,000	Total project budget	
0	Existing systems being updated or replaced	$125	BA blended hourly cost	
5	Stakeholders	Standard	Type of project	
1	Interfacing Systems - small systems	5	Number of developers	
1	Interfacing Systems - medium systems	No	Is your team remote?	
1	Interfacing Systems - large systems	52	Project duration? (weeks)	
8	Process Flows	20	Requirements work duration? (weeks)	
2	BDDs			
8	Screens			
1	Reports			

Summary Total Effort Comparison				
	% of total project	Ratio of dev to BAs	Activity based	
Number of BAs	1	0.8	0.8	*Excludes time off
BA budget for requirements work	$ 113,000	$ 83,000	$ 77,000	
BA budget for project duration	$ 293,000	$ 217,000	$ 200,000	

FIGURE 19-2 Partial output from the requirements effort estimation spreadsheet.

The requirements estimation tool has three worksheet tabs. First, there is a summary where you input several project characteristics. The tool will calculate various elements of the three types of estimates. Second, there is an assumptions tab where you can adjust items that vary from the provided assumptions. The third tab provides instructions about how to use the estimation tool.

The assumptions built into this estimation tool are based on Seilevel's extensive experience with actual projects. You'll need to tweak some of the assumptions for your own organization. For example, if your BAs are either novices or especially highly experienced, some of your estimates of the time needed per activity may vary from the defaults. To tailor the tool to best suit your reality, collect some data from your own projects and modify the adjustable parameters.

Important All estimates are based on the knowledge the estimator has available at the time and the assumptions he makes. Preliminary estimates based on limited information have large uncertainties. Refine your estimates as knowledge is gained and work is completed during the project. Record your assumptions so that it's clear what you were thinking when you came up with the numbers.

Betty's in a corner

Sridhar, the project manager of a million-dollar project, approached the BA, Betty, to discuss her initial estimate regarding how long requirements development would take. In an earlier email exchange she had estimated eight weeks. Sridhar asked, "Betty, is it really going to take you eight weeks to do the requirements for our shopping portal? Surely your team can have it done in four weeks; the system is just not that complex. I mean, really, people come to the website to search for and buy products. That's it! Heck, the development manager is thinking that his team can just develop the system without any requirements at all, so that's what they're planning to do if you don't have the requirements done in four weeks."

Betty is backed into a corner here. She can give in and agree to an unreasonable four-week deadline for this large project. Or, she can push back at the risk of looking ineffective because the project is supposed to be "simple." After all, Betty isn't actually sure how long it will take her to develop an adequate set of requirements, because she doesn't yet know the size of the system. Until she begins the analysis, she doesn't know what she doesn't know.

Variations on this story are a big part of why Seilevel developed the estimation tool described in this chapter. This tool aids Betty in her stressful conversation with Sridhar. She can say, "Well, if I only have four weeks, let me show you what I CAN do." She can tweak the numbers of reports or processes for which requirements are needed. Betty can effectively timebox the requirements effort. However, it's important for Sridhar to recognize that understanding the requirements for only the tip of the iceberg can lead to unpleasant surprises further down the road.

From requirements to project plans

Because requirements are the foundation for the project's intended work, you should base estimates, project plans, and schedules on those requirements. Remember, the most important project outcome is a system that meets its business objectives, not simply one that implements all the initial requirements according to the original project plan. The requirements and plans represent the team's assessment at a specific point in time of what it will take to achieve that outcome. But the project's scope might have been off target, or the initial plans might not have been realistic or well-aligned with the objectives. Business needs, business rules, and project constraints all can change. The project's business success will be problematic if you don't update your plans to align with evolving objectives and realities.

Estimating project size and effort from requirements

Making realistic estimates of the effort and time needed to complete a project depends on many factors, but it begins with estimating the size of the product to be built. You can base size estimates on functional requirements, user stories, analysis models, prototypes, or user interface designs. Although there's no perfect measure of software size, the following are some frequently used metrics:

- The number of individually testable requirements (Wilson 1995)
- Function points (Jones 1996b; IFPUG 2010)
- Story points (Cohn 2005; McConnell 2006) or use case points (Wiegers 2006)
- The number, type, and complexity of user interface elements
- Estimated lines of code needed to implement specific requirements

Base whatever approach you choose on your experience and on the nature of the software you're developing. Understanding what the development team has successfully achieved on similar projects using similar technologies lets you gauge team productivity. After you estimate size and productivity, you can estimate the total effort needed to implement the project. Effort estimates depend on the team size (multitasking people are less productive, and more communication interfaces slow things down) and planned schedule (compressed schedules actually increase the total effort needed).

One approach is to use commercial software estimation tools that suggest various feasible combinations of development effort and schedule. These tools let you adjust estimates based on factors such as the skill of the developers, project complexity, and the team's experience in the application domain. Complex, nonlinear relationships exist between product size, effort, development time, productivity, and staff buildup time (Putnam and Myers 1997). Understanding these relationships can keep you from being trapped in the "impossible region," combinations of product size, schedule, and team size where the probability of success is extremely low.

The best estimation processes acknowledge the early uncertainty and ongoing volatility of scope. People using such a process will express each estimate as a range, not a single value. They manage the accuracy of their estimate by widening the range based on the uncertainty and volatility of the data that fed into the estimate.

Agile projects estimate scope in units of *story points*, a measure of the relative effort that will be needed to implement a particular user story. Estimates of the size of a specific story depend on the knowledge you have—and lack—about the story, its complexity, and the functionality involved (Leffingwell 2011). Agile teams measure their team's *velocity*, the number of story points the team expects to complete in a standard iteration based on its previous experience and the results from early iterations on a new project. The team members combine the size of the product backlog with velocity to estimate the project's schedule, cost, and the number of iterations required. Dean Leffingwell (2011) describes several techniques for estimating and planning agile projects in this fashion.

Important If you don't compare your estimates to the actual project results and improve your estimating ability, your estimates will forever remain guesses. It takes time to accumulate enough data to correlate some measure of software size with requirements development effort and with total project effort. The early iterations on agile projects give the team an assessment of its velocity.

Even a good estimating process will be challenged if your project must cope with requirements that customers, managers, or lawmakers frequently change. If the changes are so great that the development team can't keep up with them, they can become paralyzed, unable to make meaningful progress. Agile development methods provide another way to deal with highly volatile requirements. These methods start by implementing a relatively solid portion of the requirements, knowing up front that changes will be made later. Teams then use customer feedback on the early increments to clarify the remaining product requirements.

A goal is not the same thing as an estimate. Anytime an imposed deadline and a thoughtfully estimated schedule don't agree, negotiation is in order. A project manager who can justify an estimate based on a well-thought-out process and historical data is in a better bargaining position than someone who simply makes her best guess. The project's business objectives should guide stakeholders to resolve the schedule conflict by stretching timelines, reducing scope, adding resources, or compromising on quality. These decisions aren't easy, but making them is the only way to maximize the delivered product value.

Got an hour?

A customer once asked our software group to adapt a small program that he had written for his personal use so that his colleagues could also access it on our network. "Got an hour?" my manager asked me, giving his top-of-the-head assessment of the project's size. When I spoke with the customer and his colleagues to understand what they really had in mind, the problem turned out to be a bit larger. I spent 100 hours writing the program they were looking for. The 100-fold expansion factor suggested that my manager's initial estimate of one hour was a trifle hasty. The team should perform a preliminary exploration of requirements, evaluate scope, and judge the product size *before* anyone makes estimates or commitments.

Uncertain requirements lead to uncertain estimates. Because requirements uncertainty is unavoidable early in the project and because estimates are usually optimistic, include contingency buffers in your schedule and budget to accommodate some requirements growth (Wiegers 2007). Scope growth takes place because business needs change, users and markets shift, and stakeholders reach a better understanding of what the software can or should do. On agile projects, scope growth typically leads to adding more iterations to the development cycle. Extensive requirements growth, however, often indicates that many requirements were missed during elicitation.

Important Don't let your estimates be swayed by what you think someone else wants to hear. Your prediction of the future shouldn't change just because someone doesn't like it. Too large a mismatch in predictions indicates the need for negotiation, though.

Requirements and scheduling

Many projects practice "right-to-left scheduling": a delivery date is cast in concrete and then the product's requirements are defined. In such cases, it often proves impossible to meet the specified ship date while including all the demanded functionality at the expected quality level. It's more realistic to define the software requirements *before* making detailed plans and commitments. A design-to-schedule strategy can work if the project manager can negotiate what portion of the desired functionality can fit within the schedule constraints. Requirements prioritization is a key success factor.

For complex systems in which software is only a part of the final product, project managers generally establish high-level schedules after developing the product-level (system) requirements and a preliminary architecture. At this point, the key delivery dates can be established, based on input from sources including marketing, sales, customer service, and development.

Consider planning and funding the project in stages. An initial requirements exploration stage will provide enough information to let you make realistic plans and estimates for one or more construction stages. Projects that have uncertain requirements benefit from incremental and iterative development approaches. Incremental development lets the team begin delivering useful software long before the requirements become fully clear. Prioritization of requirements dictates the functionality to include in each construction timebox.

Software projects frequently fail to meet their goals because the developers and other project participants are optimistic estimators and poor planners, not because they're poor software engineers. Typical planning mistakes include overlooking common tasks, underestimating effort or time, failing to account for project risks, and not anticipating rework (McConnell 2006). Effective project scheduling requires the following elements:

- Estimated product size
- Known productivity of the development team, based on historical performance
- A list of the tasks needed to completely implement and verify a feature or use case
- Reasonably stable requirements, at least for the forthcoming development iteration
- Experience, which helps the project manager adjust for intangible factors and the unique aspects of each project

Trap Don't succumb to pressure to make commitments that you know are unachievable. This is a recipe for a lose-lose outcome.

From requirements to designs and code

The boundary between requirements and design is not a sharp line but a gray, fuzzy area (Wiegers 2006). Try to keep requirements free from implementation bias, except when there's a compelling reason to intentionally constrain the design. Ideally, the descriptions of what the system is intended to do should not be slanted by design considerations. Practically speaking, though, projects often possess design constraints from prior products, product line standards, and user interface conventions. Because of this, a requirements specification almost always contains some design information. Try to avoid *inadvertent* design, needless or unintended restrictions on the design. Include designers in requirements reviews to make sure the requirements can serve as a solid foundation for design.

Architecture and allocation

A product's functionality, quality attributes, and constraints drive its architecture design (Bass, Clements, and Kazman 1998; Rozanski and Woods 2005). Analyzing a proposed architecture helps the analyst to verify the requirements and tune their precision, as does prototyping. Both methods use the following thought process: "If I understand the requirements correctly, this approach I'm reviewing is a good way to satisfy them. Now that I have a preliminary architecture (or a prototype) in hand, does it help me better understand the requirements and spot incorrect, missing, or conflicting requirements?"

Architecture is especially critical for systems that include both software and hardware components and for complex software-only systems. An essential step is to allocate the high-level system requirements to the various subsystems and components. An analyst, system engineer, or architect decomposes the system requirements into functional requirements for both software and hardware subsystems. Requirements trace information lets the development team track where each requirement is addressed in the design.

Inappropriate allocation decisions can result in the software being expected to perform functions that should have been assigned to hardware components (or the reverse), in poor performance, or in the inability to replace one component easily with an improved version. On one project, the hardware engineer blatantly told my group that he expected our software to overcome all limitations of his hardware design! Although software is more malleable than hardware, engineers shouldn't use that flexibility as a reason to skimp on hardware design. Take a systems engineering approach to decide which capabilities each system component should deliver.

Allocation of system capabilities to subsystems and components must be done from the top down (Hooks and Farry 2001). Consider a Blu-ray Disc player. As illustrated in Figure 19-3, it includes motors to open and close the disc tray and to spin the disc, an optical subsystem to read the data on the disc, an image-rendering subsystem, a multifunction remote control, and more. The subsystems interact to control the behavior that results when, say, the user presses a button on the remote control to open the disc tray while the disc is playing. The system requirements drive the architecture design for such complex products, and the architecture influences the requirements allocation.

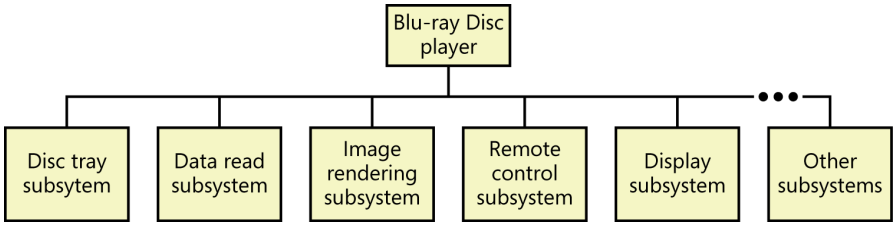

FIGURE 19-3 Complex products such as Blu-ray Disc players contain multiple software and hardware subsystems.

The incredible shrinking design

I once worked on a project that simulated the behavior of a photographic system with eight computational processes. After working hard on requirements analysis, the team was eager to start coding. Instead, we took the time to create a design model to think about how we'd build a solution. We quickly realized that three of the steps in the photographic simulation used identical computational algorithms, three more used another set, and the remaining two shared a third set. The design perspective simplified the problem from eight sets of complex calculations to just three. Had we skipped design, we likely would have noticed the code repetition at some point, but we saved a lot of time by detecting these simplifications early on. It's more efficient to revise design models than to rewrite code.

Software design

Software design receives short shrift on some projects, yet the time spent on design is an excellent investment. A variety of software designs will satisfy most products' requirements. These designs will vary in their performance, efficiency, robustness, and the technical methods employed. If you leap directly from requirements into code, you're essentially designing the software mentally and on the fly. You come up with *a* design but not necessarily with *an excellent* design. Poorly structured software is the likely result.

As with requirements, excellent designs result from iteration. Make multiple passes through the design to refine your initial concepts as you gain information and generate additional ideas. Shortcomings in design lead to products that are difficult to maintain and extend and that don't satisfy the customer's performance, usability, and reliability objectives. The time you spend translating requirements into designs is an excellent investment in building high-quality, robust products.

A project that's applying object-oriented development methods might begin with object-oriented analysis of requirements, using class diagrams and other UML models to represent and analyze requirements information. A designer can elaborate these conceptual class diagrams, which are free of implementation specifics, into more detailed object models for design and implementation.

You needn't develop a complete, detailed design for the entire product before you begin implementation, but you should design each component before you code it. Formal design is of most

benefit to particularly difficult projects, projects involving systems with many internal component interfaces and interactions, and projects staffed with inexperienced developers (McConnell 1998). All projects, however, will benefit from the following strategies:

- Developing a solid architecture of subsystems and components that will permit enhancement over the product's life
- Identifying the key functional modules or object classes you need to build, as well as defining their interfaces, responsibilities, and collaborations with other units
- Ensuring that the design accommodates all the functional requirements and doesn't contain unnecessary functionality
- Defining each code unit's intended functionality, following the sound design principles of strong cohesion, loose coupling, and information hiding (McConnell 2004)
- Ensuring that the design addresses exception conditions that can arise
- Ensuring that the design will achieve stated performance, security, and other quality goals
- Identifying any existing components that can be reused
- Defining—and respecting—any limitations or constraints that have a significant impact on the design of the software components

As developers translate requirements into designs and code, they'll encounter points of ambiguity and confusion. Ideally, developers can route these issues back to customers or BAs for resolution through the project's issue-tracking process. If an issue can't be resolved immediately, any assumptions, guesses, or interpretations that a developer makes should be documented and reviewed with customer representatives.

User interface design

User interface design is an extensively studied domain that goes well beyond the scope of this book. Your requirements explorations probably took at least tentative steps into UI design. UI design is so closely related to requirements that you shouldn't just push it downstream to be done without end-user engagement. Chapter 15, "Risk reduction through prototyping," described how use cases lead to dialog maps, wireframes, or prototypes, and ultimately into detailed UI designs. A display-action-response (DAR) model is a useful tool for documenting the UI elements that appear in screens and how the system responds to user actions (Beatty and Chen 2012). A DAR model combines a visual screen layout with tables that describe the elements on the screen and their behaviors under different conditions. Figure 19-4 shows a sample page from a website, and Figure 19-5 shows a corresponding DAR model. The DAR model contains enough details about the screen layout and behavior that a developer should be able to implement it with confidence.

FIGURE 19-4 High-fidelity webpage design.

UI Element: Submit a Pearl Page at PearlsFromSand.com	
UI Element Description	
ID	submit.html
Description	Page where users can submit their own life lessons to be posted on the Pearls from Sand blog
UI Element Description	
Precondition	**Display**
Always	"Home" link "About the Book" link "About the Author" link "Blog" link "Submit a Pearl" link (inactive, different color because it's the current page) "Buy the Book" link "Contact" link "Name" text field "City" text field "State or Province" drop-down list "Email" text field "Title" text field "Pearl Category" drop-down list "Your Story" text field "I agree" check box, cleared "Submit" button "Pearl Submission Guidelines" link "Pearl Submission Terms" link
User just submitted a pearl	"Name," "City," "State or Province," and "Email" fields are populated with values from previous pearl. "Title," "Pearl Category," "Your Story," and "I agree" fields are reset to default values.

UI Element Behaviors		
Precondition	**User Action**	**Response**
Always	User clicks on navigation links: "Home," "About the Book," "About the Author," "Buy the Book," "Contact," "Pearl Submission Guidelines," "Pearl Submission Terms"	Corresponding page is displayed
Always	User clicks on either "Blog" link	Pearls from Sand blog opens in new browser tab
Always	User types or pastes text into a text field	User's text is displayed in field; for "Your Story" field, count of remaining characters is displayed
Always	User clicks on "I agree" check box	Check box toggles on/off
One or more invalid entries	User clicks on "Submit" link	Error message appears for any invalid text entry or length or for required fields that are blank
All fields have valid entries; "I agree" check box is selected	User clicks on "Submit" link	Pearl is submitted; pearl counter is incremented; email with pearl info is sent to Submitter and Administrator; successful submission acknowledgment message is displayed.
"I agree" box not checked	User clicks on "Submit" link	System displays error message on this page

FIGURE 19-5 Display-action-response (DAR) model for the webpage shown in Figure 19-4.

From requirements to tests

Requirements analysis and testing fit together beautifully. As consultant Dorothy Graham (2002) points out, "Good requirements engineering produces better tests; good test analysis produces better requirements." The requirements provide the foundation for system testing. The product should be tested against what it was intended to do as recorded in the requirements documentation, not against its design or code. System testing that's based on the code can become a self-fulfilling prophecy. The product might correctly exhibit all the behaviors described in tests based on the code, but that doesn't mean that it meets the customers' needs. Include testers in requirements reviews to make sure the requirements are verifiable and can serve as the basis for system testing.

Agile development teams typically write acceptance tests in lieu of precise requirements (Cohn 2004). Rather than specifying the capabilities the system must exhibit or the actions a user must be able to take, the acceptance tests flesh out the expected behavior of a user story. This conveys to developers the information they need to feel confident that they've correctly and completely implemented each story. As described in Chapter 17, "Validating the requirements," acceptance tests should cover:

- Expected behavior under normal conditions (good input data and valid user actions).
- How anticipated error conditions and expected failure scenarios should be handled (bad input data or invalid user actions).
- Whether quality expectations are satisfied (for example, response times, security protections, and the average time or number of user actions needed to accomplish a task).

What to test?

A seminar attendee once said, "I'm in our system testing group. We don't have written requirements, so we have to test what we think the software is supposed to do. Sometimes we're wrong, so we have to ask the developers what the software does and test it again."

Testing what the developers built isn't the same as testing what they were *supposed* to build. The requirements are the ultimate reference for system and user acceptance testing. If the system has poorly specified requirements, the testers will discover many requirements that developers inferred—rightly or wrongly—and implemented. The analyst should document legitimate implied requirements and their origins to make future regression testing more effective.

The testers or quality assurance staff should determine how they'd verify the implementation of each requirement. Possible methods include:

- Testing (executing the software to look for defects)
- Inspection (examining the code to ensure that it satisfies the requirements)
- Demonstration (showing that the product works as expected)
- Analysis (reasoning through how the system should work under certain circumstances)

Connecting testing back to requirements helps keep the testing effort prioritized and focused for maximum benefit. One colleague, a seasoned project manager and business analyst, related her experience along these lines: "A clearly articulated business need can drive user acceptance testing (UAT), which is typically the final hurdle a project undergoes prior to going live. On a recent web portal development project, we worked with the business sponsor to understand the real gains the web portal was expected to deliver. Understanding the critical requirements allowed the project manager to craft clear definitions of critical, moderate, and cosmetic defects. By tying defect criteria clearly to requirements, we guided our customers through UAT and successfully completed a major development effort without any ambiguity about quality or acceptance criteria."

The simple act of thinking about how you'll verify each requirement is a useful quality practice. Use analytical techniques such as cause-and-effect graphs to derive tests based on the logic described in a requirement. This will reveal ambiguities, missing or implied *else* conditions, and other problems. Each functional requirement should map to at least one test so that no expected system behavior goes unverified. Requirements-based testing applies several test design strategies: action-driven, data-driven (including boundary value analysis and equivalence class partitioning), logic-driven, event-driven, and state-driven (Poston 1996). Skillful testers will augment requirements-based testing with additional testing based on the product's history, intended usage scenarios, overall quality characteristics, service level agreements, boundary conditions, and quirks.

The effort invested in early test thinking isn't wasted, even if you plan a separate system testing effort before release. It's a matter of reallocating test effort that historically was weighted toward the latter project stages. Conceptual tests are readily transformed into specific test scenarios and automated, where feasible and appropriate. Moving test thinking up earlier in the development cycle will pay off with better requirements, clear communication and common expectations among stakeholders, and early defect removal.

As development progresses, the team will elaborate the requirements from the high level found in user requirements, through the functional requirements, and ultimately down to specifications for individual code modules. Testing authority Boris Beizer (1999) points out that testing against requirements must be performed at every level of software construction, not just the end-user level. Some application code isn't directly accessed by users but is needed for infrastructure operations. Each module must satisfy its own specification, even if that module's function is invisible to the user. Consequently, testing the system against user requirements is a necessary—but not sufficient—strategy for system testing.

From requirements to success

I once encountered a project in which a contract development team came on board to implement a very large application for which an earlier team had developed the requirements. The new team took one look at the dozen three-inch binders of requirements, shuddered in horror, and began coding. They didn't refer to the SRS during construction. Instead, they built what they thought they were supposed to build, based on an incomplete and inaccurate understanding of the project's goals. Not surprisingly, this project encountered a lot of problems. Trying to understand a huge volume of even excellent requirements is certainly hard, but ignoring them is a decisive step toward project failure.

It's faster to read the requirements, however extensive, before implementation than it is to build the wrong system and then have to build it again correctly. It's even faster to engage the development team early in the project so that they can participate in the requirements work and perform early prototyping or take an iterative development approach. The development team still has to read the entire specification eventually. However, they are spreading their reading time across the project, which alleviates some of the tedious nature of the activity.

A more successful team had a practice of listing all the requirements that were planned for a specific release. The project's quality assurance group evaluated each release by executing the tests for those requirements. A requirement that didn't satisfy its test criteria was counted as a defect. The QA group rejected the release if more than a predetermined number of requirements weren't met or if specific high-impact requirements weren't satisfied. This project was successful largely because it used its documented requirements to decide when a release was shippable.

The ultimate deliverable from a software development project is a solution that meets the customers' needs and expectations. Requirements are an essential step on the path from business need to satisfied customers. If you don't base your project plans, designs, and acceptance and system tests on a foundation of high-quality requirements, you're likely to waste a great deal of effort trying to deliver a solid product. Don't become a slave to your requirements processes, though. There's no point in spending time generating unnecessary documents and holding ritualized meetings. Strive for a sensible balance between rigorous specification and off-the-top-of-the-head coding that will reduce the risk of building the wrong product to an acceptable level.

Next steps

- Estimate the requirements work on your next project by using the requirements estimation tool from Figure 19-2. Track your time on the project and compare the results to your initial estimation. Adapt the estimation tool for your next project.
- Estimate the percentage of unplanned requirements growth on your last several projects. Can you build contingency buffers into your project schedules to accommodate a similar scope increase on future projects? Use the growth data from previous projects to justify the schedule contingency so that it doesn't look like arbitrary padding.
- Try to trace all the requirements in an implemented portion of your SRS to individual design elements. The design elements might be processes in design data flow diagrams, tables in data models, object classes or methods, or other design components. Are any design elements missing? Were any requirements overlooked?
- Record the number of lines of code, function points, story points, or UI elements that are needed to implement each feature or user requirement. Also record the actual effort needed to fully implement and verify each feature or use case. Look for correlations between size and effort that will help you make more accurate estimates in the future.
- Record your estimates of size and effort for the requirements development activities and deliverables on your project, and compare those to the actual results. Did you really do the 5 interviews planned, or did you end up doing 15? Did you create twice as many use cases as expected? How can you change your estimation process to be more accurate in the future?

PART III

Requirements for specific project classes